

A Systematic Literature Review of I/O Stack Evolution for Fully Utilizing SSDs

Dawen Liang

College of Engineering and Computer Science
Syracuse University
Syracuse, NY, USA
dliang02@hotmail.com

Hengbo Cai

College of Engineering and Computer Science
Syracuse University
Syracuse, NY, USA
chb59468@hotmail.com

Abstract—The rise of ultra-low latency (ULL) NVMe SSDs, which have device access times nearing single-digit microseconds, has drastically changed the performance bottleneck from storage hardware to the software I/O stack. This systematic literature review examines how the I/O stack has evolved—from kernel-based mediation to hybrid and full kernel bypass solutions—to fully harness the capabilities of modern SSDs. Following PRISMA 2020 guidelines [1], we gathered 3,131 records from four databases and via citation-based snowballing, eventually including 398 studies covering the period from 2012–2025. We organize the reviewed work into a three-path taxonomy: *Kernel Path* optimizations (multi-queue block layer, `io_uring`, polling, and file system adaptations), *Hybrid Approaches* (eBPF/XRP in-kernel bypass, NVMe I/O Passthrough, and modernized FUSE), and *Full Kernel Bypass* solutions (SPDK, user-space file systems, and interrupt-based user-space stacks). Key findings include: (1) the Linux kernel storage path contributes up to 50% of end-to-end I/O latency when using modern NVMe devices; (2) promising optimizations in the kernel path, like `blk-switch`, achieve an average latency improvement of up to $130\times$ over standard Linux; (3) hybrid approaches such as XRP and I/O Passthru deliver throughput gains of 30–94% while maintaining kernel safety guarantees; and (4) full bypass platforms continue to advance beyond SPDK toward energy-efficient and interrupt-based designs. We identify convergence trends—particularly the blurring of kernel and bypass boundaries through eBPF programmability and NVMe passthrough—and outline open challenges including multi-tenancy, security, and energy efficiency for upcoming I/O stack research.

Index Terms—I/O stack, block layer, ultra-low latency SSD, NVMe, kernel bypass, `io_uring`, SPDK, eBPF, systematic literature review

I. INTRODUCTION

The performance of solid-state drives (SSDs) has seen significant improvements over the last decade. Modern NVMe SSDs offer random read latencies below $10\mu\text{s}$ and sequential bandwidths above 7 GB/s, approaching the throughput limits of the PCIe interconnect [2], [3]. This rapid hardware advancement challenges a long-held assumption in systems design: for ultra-low latency (ULL) SSDs, the *software* I/O stack—not the storage

device—has become the dominant performance bottleneck.

Barroso et al. identified this paradigm shift as the “attack of the killer microseconds,” observing that neither nanosecond-scale hardware techniques (branch prediction, prefetching) nor millisecond-scale software techniques (context switching, thread-based concurrency) are effective for microsecond-regime operations [4]. Enberg et al. further demonstrated that hardware queue constraints in modern processors (e.g., at most 10 Line Fill Buffers per core) impose fundamental limits on prefetch-based latency hiding, while application-managed queues incur management overhead that caps peak performance at 50% of the DRAM baseline [5].

The traditional Linux I/O stack—comprising the Virtual File System (VFS), file system, page cache, block layer, and device driver—was designed when storage devices operated at millisecond latencies [6]. Fig. 1 illustrates this layered architecture, where each I/O request traverses the system call interface, file system, block layer (with `blk-mq` per-core queues), and device driver before reaching the hardware [7]–[9]. With ULL SSDs, these software layers introduce overheads that were previously negligible: context switches, data copies between kernel and user buffers, interrupt handling, I/O scheduling, and lock contention.

The asynchronous I/O stack by Lee et al. demonstrated that overlapping kernel operations with device I/O can reduce full-path latency to below $10\mu\text{s}$ for 4 KB random reads on Intel Optane SSDs—a 15–33% improvement over the traditional path [10]. Yet even with such optimizations, the kernel’s contribution to end-to-end latency remains substantial. As Fig. 2 illustrates, Storage Class Memory (SCM) devices such as Intel Optane, Samsung Z-SSD, and KIOXIA XL-Flash bridge the latency gap between DRAM ($\sim 100\text{ns}$) and conventional NVMe SSDs ($150\text{--}250\mu\text{s}$) with access times around $15\mu\text{s}$, further exacerbating the software overhead problem. Fig. 3 confirms this quantitatively: for ULL devices such as Z-SSD and Optane, the kernel accounts

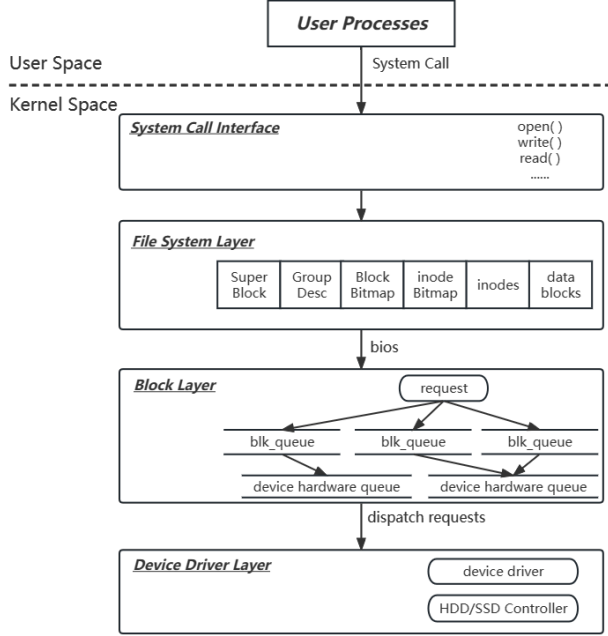


Fig. 1. The traditional Linux I/O stack. Each I/O request from user space traverses multiple kernel layers—system call interface, file system, block layer (with blk-mq per-core queues), and device driver—before reaching the storage device [7]–[9].

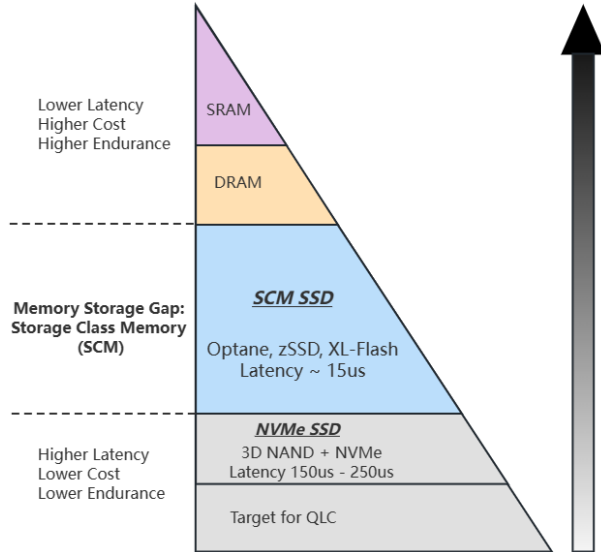


Fig. 2. Storage Class Memory (SCM) in the memory–storage hierarchy. SCM SSDs (Optane, Z-SSD, XL-Flash) with $\sim 15\mu s$ latency bridge the gap between DRAM and conventional NVMe SSDs, making software I/O stack overhead the dominant bottleneck.

for 24–36% of total I/O latency, while for 4 KB random writes with `fsync`, kernel overhead reaches 29–36% even on NVMe SSDs [9].

In response, the research community has explored three broad architectural strategies. *Kernel path opti-*

mizations retain the existing Linux I/O architecture but reduce overhead through multi-queue block layers (blk-mq) [11], advanced scheduling [12], and asynchronous interfaces (io_uring) [13]. *Hybrid approaches* selectively bypass specific kernel layers while preserving others, exemplified by eBPF-based storage functions (XRP) [14], NVMe I/O Passthrough [15], and modernized user-space file system frameworks (RFUSE) [16]. *Full kernel bypass* solutions remove the kernel entirely from the data path, led by Intel’s SPDK [17] and user-space file systems such as uFS [18] and NVMeDirect [19].

Despite this rich body of work, no existing survey systematically covers the full spectrum of I/O stack evolution from 2012 to 2025 using a reproducible methodology. Prior surveys have addressed specific aspects—Didona et al. [13] compared storage APIs, and Koh et al. [2] characterized ULL SSD performance—but a comprehensive, PRISMA-compliant review spanning kernel, hybrid, and bypass approaches is lacking.

This paper addresses that gap. We conducted a systematic literature review following PRISMA 2020 guidelines [1], identifying 398 relevant studies from a pool of 3,131 records. Our contributions are:

- A reproducible, PRISMA-compliant search and screening methodology across four databases plus snowballing, yielding a corpus of 398 included studies with quality assessment.
- A three-path taxonomy (Kernel, Hybrid, Full Bypass) that organizes the surveyed work and reveals convergence trends.
- A technical synthesis of 44 deeply analyzed papers covering innovations from blk-mq (2013) to interrupt-based user-space stacks (2025).
- Identification of open challenges and future directions for I/O stack research in the era of sub-microsecond storage.

The remainder of this paper is organized as follows. Section II describes our research methodology. Section III presents the search and screening results. Section IV provides the technical survey organized by the three-path taxonomy. Section V discusses cross-cutting themes and implications. Section VI outlines future research directions, and Section VII concludes.

II. RESEARCH METHODOLOGY

We performed a systematic literature review in line with PRISMA 2020 guidelines [1]. Our methodology encompasses five phases: formulating research questions, conducting a systematic search, two-stage screening, quality assessment, and data extraction. This review covers research published between 2012 and May 2025.

A. Research Questions

Drawing on the PICOC framework (Population: ULL NVMe SSDs; Intervention: I/O stack software optimiza-

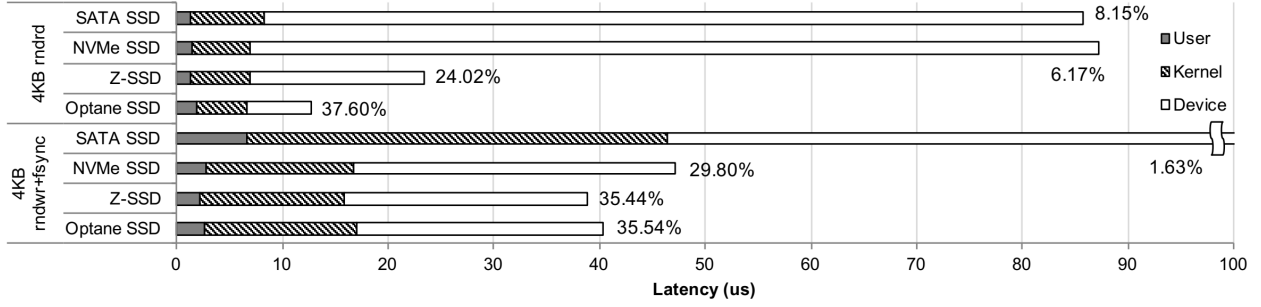


Fig. 3. I/O latency breakdown across storage devices. Percentages indicate the kernel’s share of total latency. As device speed increases from SATA SSD to Optane SSD, the kernel’s relative contribution grows from 8% to 38% for 4 KB random reads. Source: adapted from [9].

tion; Comparison: traditional Linux block layer / POSIX I/O; Outcomes: latency, throughput, CPU efficiency; Context: OS kernel and system software), we formulated four research questions:

- **RQ1:** How have I/O stack design principles changed from traditional HDD-based systems to modern SSD-focused solutions?
- **RQ2:** Which traditional I/O stack principles are still valid for ULL SSDs, and how have they been modified?
- **RQ3:** What new principles and methods have appeared to meet the performance and scalability needs of ULL SSDs?
- **RQ4:** What future research directions and challenges must be confronted to further optimize I/O stacks for high-performance storage?

B. Search Strategy

We searched four electronic databases and supplemented with citation-based snowballing (Table I). Five Boolean query strings were created to cover the main I/O stack optimization technologies (Table II). Each query was executed against DB1–DB4 with the search cutoff set to May 31, 2025. For DB5, we performed forward and backward citation tracing on six seed papers, including seminal works on SPDK [17], the asynchronous I/O stack [10], and FlashShare [8].

Additionally, 80 papers from the first author’s prior review work were included as a supplementary source. These papers had been independently curated through an earlier search process and entered the current review at the full-text assessment stage (Stage 2), where they were subjected to the same eligibility criteria as database-sourced records.

C. Inclusion and Exclusion Criteria

Studies were included if they: (a) examine I/O stack software optimizations aimed at NVMe or ULL SSDs; (b) present designs, implementations, or evaluations

TABLE I
SEARCH SOURCES

ID	Source	Type
DB1	Google Scholar	General academic
DB2	ACM Digital Library	CS-specific
DB3	IEEE Xplore	CS/EE-specific
DB4	Semantic Scholar (API)	Multidisciplinary
DB5	Forward/backward snowballing	Citation tracing

TABLE II
QUERY STRINGS

QID	Query String
Q1	“io_uring” OR “SPDK” OR “blk-mq” NVMe SSD performance
Q2	“kernel bypass” NVMe storage latency
Q3	eBPF storage “I/O stack” OR “storage stack” NVMe
Q4	“ultra-low latency” SSD “I/O” optimization kernel
Q5	NVMe “user-space” OR “userspace” driver file system

of kernel-path, hybrid, or full kernel-bypass I/O approaches; or (c) provide empirical performance data in the context of modern SSD storage.

Exclusion was governed by five mutually exclusive criteria applied in strict priority order (EC1 > EC2 > EC3 > EC4 > EC5):

Scope boundary: Research on CXL (Compute Express Link), ZNS-only optimizations lacking contributions to the I/O stack, KV-SSD interfaces, and computational storage offloading was excluded under EC3.

D. Screening Process

After combining records from all sources, 710 duplicates were identified by DOI match and normalized title similarity, leaving 2,341 unique records.

Stage 1 (title and abstract screening) excluded 1,849 records: EC1: 39, EC2: 18, EC3: 1,677, EC4: 115. The

TABLE III
EXCLUSION CRITERIA

Code	Definition
EC1	Non-computer-science domain
EC2	Pure hardware (no software stack contribution)
EC3	Not focused on ULL I/O stack optimization
EC4	Non-peer-reviewed (with critical exceptions)
EC5	Duplicate contribution (superseded version)

TABLE IV
QUALITY ASSESSMENT DISTRIBUTION

Score	Criteria	Count	%	Role
3	Top-tier venue + artifacts	72	18.1	Primary evidence
2	Recognized venue + solid eval	65	16.3	Supporting
1	Workshop/short/limited eval	52	13.1	Supplementary
0	Minimal evaluation	209	52.5	Audit only

high number of EC3 exclusions reflects a broad search strategy aimed at maximum recall.

Stage 2 (full-text eligibility) assessed 572 reports (492 from Stage 1 plus 80 prior-review papers) and excluded 174: EC1: 105, EC2: 17, EC3: 48, EC4: 2, EC5: 2. The final included corpus comprises **398 studies**.

The arithmetic checksum verifies: $398 + 174 + 1,849 + 710 = 3,131$.

E. Quality Assessment

Each included study was scored on a 0–3 quality scale:

The 209 studies scored QA=0 remain in the included count for audit transparency but are excluded from the technical synthesis. Sections IV and V draw exclusively from the 189 studies scoring $QA \geq 1$.

F. Data Extraction and Classification

For each study, we gathered bibliographic metadata, Path_Type classification (*Kernel*, *Hybrid*, or *Full Bypass*), key technologies used, optimization techniques, and reported performance metrics. The Path_Type taxonomy serves as the main organizational structure for Section IV.

III. SEARCH RESULTS

This section reports the outcome of the systematic search and screening process and presents descriptive statistics of the included corpus.

A. PRISMA Flow

A total of 3,131 records were identified: 3,051 from database searches (DB1–DB5) and 80 from the first author’s prior review work. After removing 710 duplicates, 2,341 unique records entered Stage 1 screening. Title and abstract screening excluded 1,849 records, with most of these (1,677; 90.7% of Stage 1 exclusions) removed under EC3. The remaining 492 records were combined with the 80 prior-review papers for Stage 2 full-text assessment, during which 174 reports were excluded. The final included corpus comprises 398 studies (Fig. 4).

B. Publication Year Distribution

The corpus spans over a decade of I/O stack research, with a clear increase in publication volume from 2019 onward. The 2012–2015 period (16 papers, 4.0%) includes foundational work like blk-mq and early polling studies. The 2016–2018 period (50 papers, 12.6%) saw the growth of NVMeDirect, FlashShare, and SPDK. The most productive period is 2019–2021 (74 papers, 18.6%), which includes the debut of io_uring, the asynchronous I/O stack, and XRP. The 2022–2025 period (135 papers, 33.9%) introduced significant designs such as I/O Passthru, RFUSE, BypassD, and Aeolia. An additional 123 papers (30.9%) from curated collections lack complete year metadata.

C. Venue Distribution

Among the 240 papers with identifiable venue metadata, USENIX venues (FAST, ATC, OSDI, HotStorage) account for 47 papers, reflecting the systems research community’s sustained focus on optimizing the storage stack. Conference proceedings dominate (153 papers), followed by journal articles (54), preprints (11), and other venues (22).

D. Source Category Breakdown

The 398 included studies consist of: DRAFT3 (9 seed papers hand-selected for the initial manuscript), GOLD (271 papers from the author’s curated collection independently confirmed by DB1–DB5), GOLD-PRIOR (80 papers from prior work not captured by current searches), and NEW-REVIEW (38 newly discovered 2024–2025 papers). The GOLD category (68.1%) shows significant overlap between the systematic search and prior work, providing confidence in search recall.

IV. TECHNICAL SURVEY OF I/O STACK EVOLUTION (2013–2025)

This section synthesizes the technical contributions of 44 deeply analyzed papers, organized along the three-path taxonomy shown in Fig. 5: Kernel Path optimizations (§IV-A), Hybrid Approaches (§IV-B), and Full Kernel Bypass (§IV-C). We also discuss cross-cutting concerns (§IV-D) that span multiple paths.

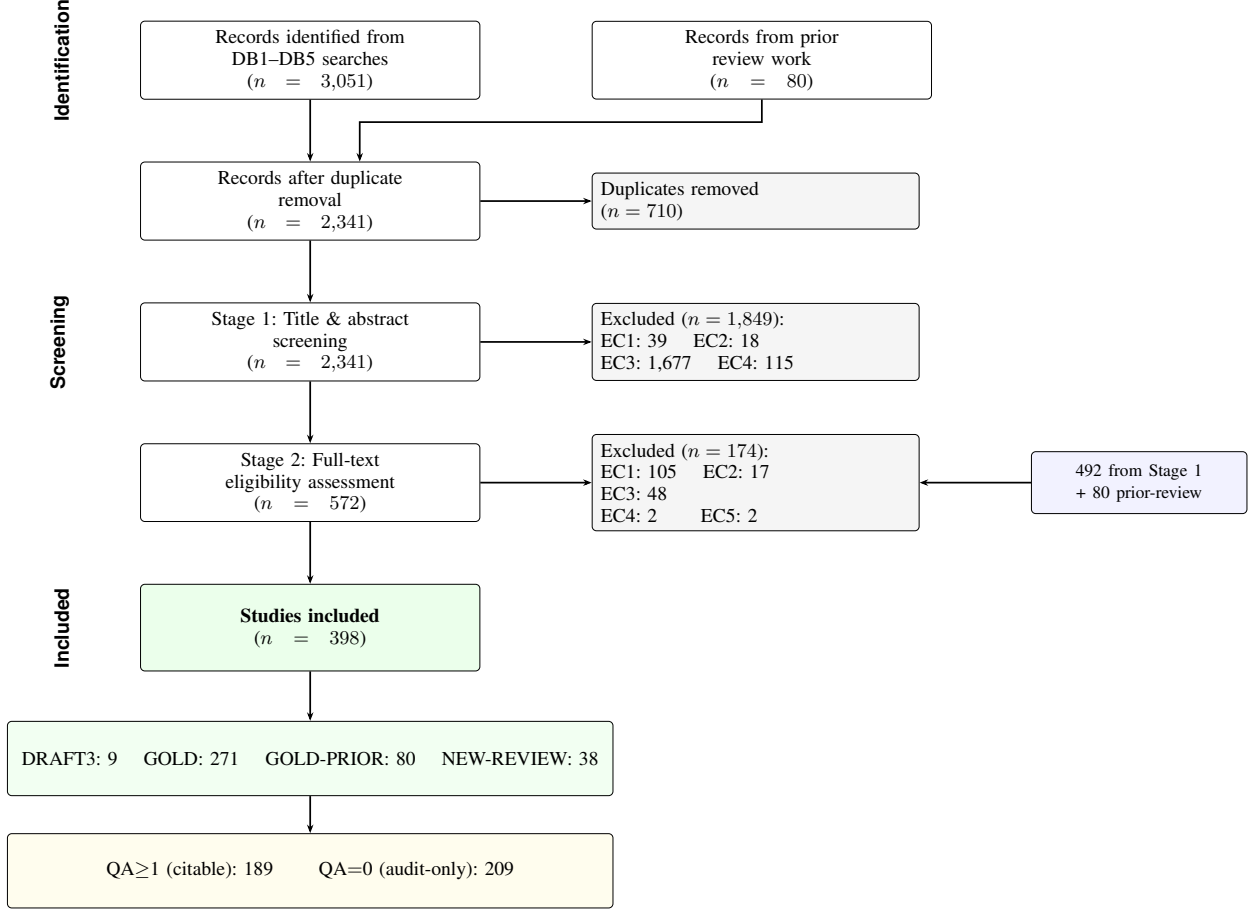


Fig. 4. PRISMA 2020 flow diagram. EC1–EC5 denote the exclusion criteria defined in Table III. The 80 prior-review papers enter at Stage 2 after independent confirmation. The arithmetic checksum verifies: $398 + 174 + 1,849 + 710 = 3,131$.

A. Kernel Path Optimizations

Kernel path optimizations keep the Linux I/O stack architecture but systematically reduce overhead at each layer. We group these into four areas: block layer improvements, asynchronous and memory-mapped I/O, file system changes, and zoned storage interfaces.

1) *Block Layer: Multi-Queue and Scheduling*: The introduction of the multi-queue block layer (blk-mq) by Bjørling et al. in 2013 marked a foundational change [11]. It replaced the single-queue block layer with per-core software queues that map to hardware submission and completion queues in NVMe devices. This design removed a major bottleneck and allowed the block layer to scale with modern multi-core, multi-queue hardware.

Despite improvements from blk-mq, head-of-line blocking between latency-sensitive and throughput-oriented requests remained a critical issue in multi-tenant environments. Hwang et al. tackled this with blk-switch [7], which applies network switch design principles to the storage stack. The key insight is

that Linux’s per-core block layer queues, alongside multi-queue hardware, make the storage stack similar to a multi-ingress, multi-egress network switch. blk-switch uses this similarity to implement request-level and thread-level load balancing across cores, achieving up to $130\times$ better average latency and $24\times$ better P99 latency over baseline Linux while maintaining 84–100% of throughput. Notably, blk-switch also outperforms SPDK in tail latency by up to $12\times$ and $18\times$ for average and P99 latency, respectively, showing that kernel-path methods can compete with full bypass in mixed workloads.

I/O scheduling overhead has also been examined for ULL devices. Lee et al. proposed D2FQ (Device-Direct Fair Queueing), which combines I/O submission and dispatch into a single step by using the NVMe Weighted Round Robin (WRR) arbitration mechanism on the device side [12]. D2FQ saves up to 45% of CPU cycles compared to the latest MQFQ scheduler while reducing I/O latency by 35% and increasing bandwidth by 54%. This device-direct approach finds an important middle ground between traditional kernel scheduling and

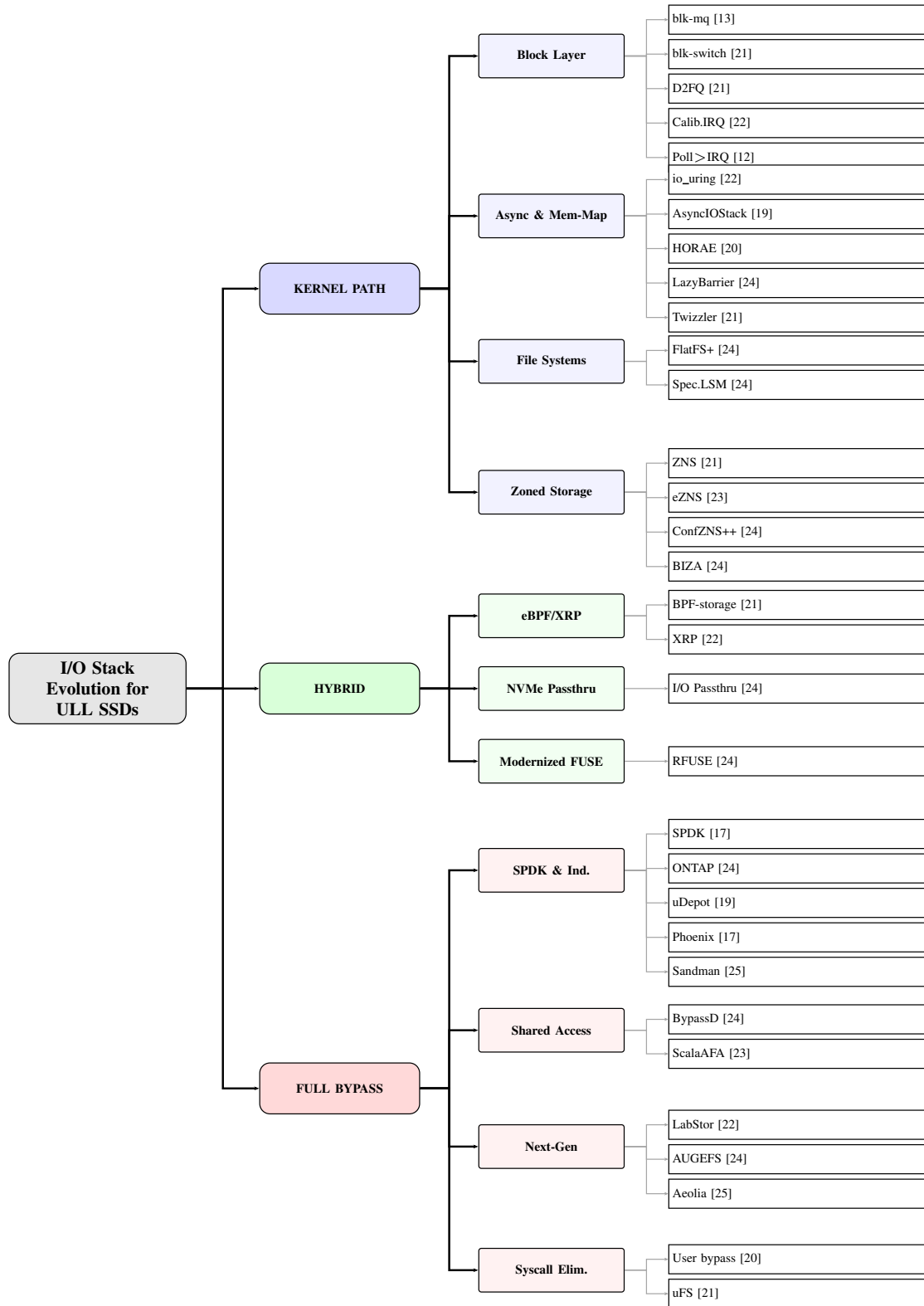


Fig. 5. Taxonomy of I/O stack optimization approaches for ULL SSDs. The 44 deeply analyzed works are organized along three architectural paths: Kernel Path (optimizations within the existing Linux I/O stack), Hybrid (selective layer bypass), and Full Bypass (complete kernel removal from the data path). Year labels indicate publication date.

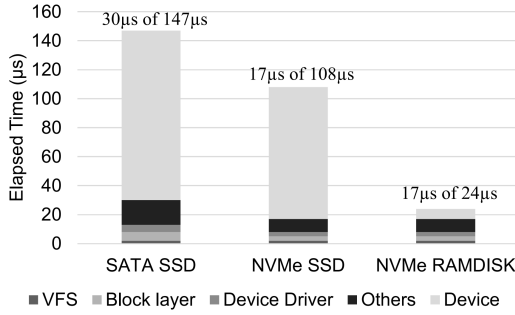


Fig. 6. Comparison of elapsed time per I/O stack layer across SATA SSD, NVMe SSD, and NVMe RAMDISK. Software overhead (VFS + Block layer + Device Driver) is 30 μ s of 147 μ s total for SATA, but 17 μ s of 24 μ s (71%) for NVMe RAMDISK, demonstrating that software becomes the dominant bottleneck as device latency decreases. Source: adapted from [24].

complete kernel bypass, allowing scheduling decisions to be made by the device hardware itself.

Harris and Altıparmak evaluated whether I/O schedulers are still beneficial for ULL devices [20]. Their findings are clear: for ULL storage, schedulers like mq-deadline and kyber add 0.5–0.9 μ s (around 6%) to median latency, while bfq can increase it by up to 2.5 μ s (20%). They concluded that I/O schedulers have become unnecessary for ULL devices when the focus is on performance or energy efficiency. However, they acknowledge that schedulers can be valuable for fairness and quality of service on slower devices.

The I/O completion mechanism—interrupts versus polling—represents another important optimization area. Yang et al. first demonstrated in 2012 that polling is better than interrupt-driven I/O for emerging fast storage devices [21]. This insight lays the groundwork for much of the later I/O stack optimization work. Building on this, Yang et al. proposed calibrated interrupts, a hybrid approach where applications explicitly mark latency-sensitive requests using a minimal two-bit device interface change [22]. The device adjusts its interrupt timing to match the completion of latency-sensitive requests, achieving up to 35% higher throughput, 30% lower CPU use, and 37% lower latency compared to traditional coalesced interrupts.

Harris and Altıparmak also showed that classic polling on ULL devices can be more energy-efficient than interrupts, contrary to common belief [23]. They found that CPU use does not directly correlate with overall system power consumption. With a single thread, classic polling achieves 8.1 μ s latency (2 μ s better than interrupts) with no context switches.

2) *Asynchronous and Memory-Mapped I/O*: The io_uring interface, introduced in Linux 5.1, offers a high-performance asynchronous I/O option through shared submission and completion queues between user space

and the kernel. This avoids the overhead of per-request system calls. Didona et al. [13] systematically compared io_uring against SPDK and libaio, concluding that io_uring can reach SPDK-level performance when properly configured with polling modes. However, it requires careful tuning and attention to CPU resource use.

The asynchronous I/O stack by Lee et al. [10] showed that specific operations within the kernel read and write paths—extent tree preloading, asynchronous page allocation, lazy page cache indexing, and lazy DMA unmapping—can overlap with device time. Combined with a lightweight block I/O layer that eliminates merging and scheduling delays, this approach achieved latencies below 10 μ s for 4 KB random reads on Optane SSDs.

Write ordering constraints add more overhead in the kernel path. Liang et al. introduced HORAE [25], which separates the control of write dependencies from the data flow by using MMIO for ordering control and the standard block layer for data flow. This architectural change avoids combining multiple hardware queues into a single logical queue, which can lead to overhead as high as 87% with increasing queue counts. HORAE delivers performance improvements of up to 1.8 $\times$ and 2.1 $\times$ in MySQL and BlueStore, respectively.

LazyBarrier [26] addresses barrier write overhead in the Android I/O stack, where `fsync()` achieves only 3.9% of buffered write throughput. Its “Lazy Issue” mechanism shares global and greedy barrier write command (BWC) across threads, paired with per-hardware-queue FIFO scheduling. This results in 73% higher throughput than the no-barrier baseline (dual-thread) and reduces latency from 51 μ s to 20 μ s. This shows that smart kernel algorithms can yield significant improvements without needing kernel bypass.

Memory-mapped I/O has also been re-examined for fast storage. Papagiannis et al. [27] identified scaling issues in the Linux mmap path for quick devices and proposed optimizations such as reducing TLB shootdowns and handling page faults in parallel. Twizzler [28] takes a bolder approach, suggesting a “data-centric” operating system where persistent data is accessed through memory-mapped objects with persistent pointers. This eliminates the traditional process-centric model and its related I/O stack overhead entirely.

3) *File System Adaptations*: Managing file system metadata can be a challenge when storage systems operate with microsecond latencies. Son et al. proposed FlatFS+ [29], which replaces traditional hierarchical namespaces with a flat namespace architecture tailored for byte-addressable NVMs. This change reduces the overhead linked to metadata lookups that can slow down fast storage.

Wu et al. tackled the access path of KV-stores using speculative multi-level access in LSM tree-based stores [30]. Their method switches from synchronous I/O to asynchronous I/O for reading data blocks, allowing multiple SSTables to be accessed at the same time. With 16 client threads, their solution boosts throughput by up to 85% compared to baseline RocksDB, although it results in 9–26% over-fetched I/O requests, showing the tradeoff between performance and resources common in asynchronous optimization.

UnistorFS [31] presents a union storage file system that removes the line between memory and storage on PRAM devices. It allows interchangeable page management using a new `GFP_UNISTOR` flag in the Linux kernel’s allocation system. This method cuts down on unnecessary data movement and improves resource use by enabling memory and storage pages to function together without logical partitioning.

4) *Zoned Storage Interfaces*: The Zoned Namespace (ZNS) interface reconsiders the host–device relationship. Bjørling et al. [32] showed that the traditional block interface carries a significant “tax” for flash SSDs due to capacity over-provisioning, DRAM for page mapping tables, and garbage collection overhead. ZNS SSDs can deliver up to $2.7\times$ higher throughput than block-interface SSDs in concurrent write tasks and up to 64% lower average random read latency. Using RocksDB on f2fs/ZNS results in $2\text{--}4\times$ lower P99.9 read tail latency.

Several studies have built on ZNS to address its drawbacks. Han et al. introduced ZNS+ [33], which adds in-storage zone compaction to cut down on write amplification and reduce CPU overhead on the host. LightNVM [34] previously led the way in developing the open-channel SSD subsystem in Linux, advocating that SSD management choices should be handled by host software rather than hidden firmware.

Lee et al. presented eZNS [35], an elastic zoned namespace that tackles the limitations of fixed zone setups. eZNS incorporates zone ballooning, which adjusts zone stripe widths based on workload needs, a hardware abstraction layer that requires only three device parameters, and a smart read scheduler. This strategy draws from virtualization memory ballooning to improve resource use among co-located namespaces.

ConfZNS++ [36] highlights that even host-controlled interfaces like ZNS can introduce new kinds of I/O management interference. The study shows that host-issued I/O management tasks, such as zone resets or finishes, interfere with data I/O. It proposes two solutions: ZINC, an I/O scheduler that prioritizes data I/O over management tasks (reducing reset interference by 56.9%), and a soft-finish operation (reducing finish interference by 50.7%).

BIZA [37] closes the compatibility gap between

block interfaces and ZNS by combining a block-to-ZNS adapter with RAIZN into one unified layer. BIZA addresses the issue that current I/O stacks do not guarantee the order in which requests are submitted—a single in-flight write can lose up to 65.3% throughput with ZNS. Through intra-zone parallelism via the Zone Random Write Area (ZRWA) and coordinated garbage collection across layers, BIZA lowers write amplification by 42.7%, boosts write throughput by 93.2%, and cuts P99.99 tail latency by 62.8%.

ZMS [38] expands zone abstractions to mobile flash storage (UFS, not NVMe), achieving $5.0\text{--}13.6\times$ better random write performance and $2.9\text{--}6.4\times$ lower write amplification through cross-layer optimization involving the file system, block layer, and firmware.

B. Hybrid Approaches

Hybrid methods selectively bypass certain kernel layers while keeping others in place, creating a design space between full kernel mediation and complete bypassing. This area has seen vibrant development from 2021 to 2025, fueled by eBPF programmability, NVMe passthrough, and modern user-space frameworks.

1) *eBPF/XRP: In-Kernel Bypass*: The idea that eBPF can serve as a basis for storage optimization was first described by Zhong et al. in their HotOS position paper [39]. They indicated that the kernel storage path contributes to about half of the access latency in modern NVMe devices. An analysis of software overhead shows that ext4 accounts for 32.0%, the BIO layer for 6.0%, and kernel crossings for 8.8%.

This concept was put into practice with XRP [14], which allows applications to run user-defined storage functions (like index lookups and aggregations) from an eBPF hook in the NVMe driver, securely bypassing most of the kernel’s storage stack. Unlike complete kernel bypass, eBPF is an OS-supported tool that ensures isolation and avoids the low utilization that busy-waiting can cause. For dependent I/O chains, which are common in B-tree lookups, XRP achieves 47–94% higher throughput and 20–34% lower P99 latency than using `read()`. XRP also facilitates better core sharing than SPDK, showing a 56% improvement in P99 latency when two threads share the same core.

To maintain file system principles, XRP forwards a small amount of kernel state (file extent mappings) to the NVMe driver hook using a soft-state cache that takes advantage of the relative stability of file extents. This design ensures the safety of the file system while allowing for application-specific tweaks at a strategic point in the I/O stack.

2) *NVMe Passthrough (I/O Passthru)*: Joshi et al. [15] introduced I/O Passthru, a new I/O path in Linux that skips the block layer entirely using a character-device

interface (`/dev/ng0n1`) along with `io_uring_cmd`. The key motivation goes beyond just performance. The block interface cannot represent all NVMe features, including ZNS with non-power-of-two zone sizes, extended LBA formats, and KV command sets. I/O Passthru reaches 5M IOPS, which is 31% higher than the 3.83M IOPS of the block path, with a handler latency of only 144 ns compared to 209 ns for the block path. This method has been integrated into the Linux kernel from versions 5.13 to 6.2, representing a production-ready hybrid solution.

In practical deployments, I/O Passthru supports Flexible Data Placement (FDP), reducing write amplification from >2 to approximately 1.

3) *Modernized FUSE (RFUSE)*: The traditional FUSE framework struggles with significant performance issues due to its single shared pending queue, which creates lock contention and prevents user-space file systems from performing well. RFUSE [16] modernizes FUSE with three main improvements: per-core ring channels that replace the shared pending queue, hybrid polling ($50\ \mu\text{s}$ busy-wait then sleep), and improved path traversal. RFUSE achieves 53% lower latency and $2.27\times$ higher throughput compared to FUSE for random reads, coming close to EXT4 performance in macro-benchmarks while maintaining full API compatibility with existing FUSE file systems. This success shows that traditional systems can be updated to achieve competitive performance instead of being discarded.

4) *BPF for Storage: The Exokernel Vision*: Zhong et al.'s earlier position paper [39] outlined a broader vision for BPF in storage, inspired by exokernel file systems. The paper suggested triggering BPF functions in the NVMe driver interrupt handler for each block I/O completion. This gives those functions access to raw block buffers to determine file offsets and quickly reissue any dependent I/O tasks. This approach has led to throughput improvements of up to $2.5\times$ and latency reductions of 49% for dependent I/O chains.

A significant design challenge is maintaining file system safety guarantees, especially with address translation between physical blocks and file offsets. The authors address this by caching extent metadata at the NVMe layer.

C. Full Kernel Bypass

Full kernel bypass removes the kernel completely from the data-path I/O. This change allows user-space applications to interact directly with NVMe devices. SPDK [17] remains the leading framework. However, recent efforts address its key limitations: device sharing, energy efficiency, modularity, and the polling assumption.

1) *SPDK and Industrial Adoption*: SPDK offers user-space NVMe drivers, polled-mode application scheduling (one thread per core with lockless queues), and implementations for storage protocols. SPDK achieves around $7\ \mu\text{s}$ read latency and $8\ \mu\text{s}$ write latency for 4 KB random I/O at queue depth 1. This results in a 44% reduction in latency for reads and a 36% reduction for writes compared to the kernel stack [17].

The industrial impact of SPDK-based architectures is shown by ONTAP's evolution [40]. NetApp's ONTAP storage system, which was originally designed for spinning drives, underwent years of optimization for SSDs. The main strategy included layer bypassing: WAFL Reply Fastpath removed message-passing hops between software components, and RAID Fastpath bypassed the RAID component on the read path. These improvements cut software overhead from multiple milliseconds to under $160\ \mu\text{s}$ (more than $20\times$), increasing read throughput at an average latency of 1 ms. ONTAP's experience confirms that incremental layer bypassing for error-free data paths is an effective and safe optimization strategy for production systems.

uDepot [41] showcases the practical use of user-space I/O principles for KV stores. It is built on the TRT (Task Runtime for Taming) framework with both libaio and SPDK backends. uDepot achieves $1.95\text{--}2.1\times$ better YCSB throughput compared to Aerospike and $10.2\text{--}14.7\times$ better than ScyllaDB, demonstrating the benefits of kernel bypass, polling instead of interrupts, and minimal cross-core communication.

Phoenix [42] uses memory-centric I/O interfaces for HPC workloads. It models all I/O as operations on persistent memory objects and takes advantage of aggregate DRAM+NVM bandwidth. This approach leads to up to $12\times$ performance improvement through a redesign of the I/O stack.

Sandman [43] addresses SPDK's biggest practical limitation: 100% CPU utilization from busy-polling. It uses hardware user wait instructions (`monitorx/mwaitx`) for microsecond-level workload detection and shallow C-1 sleep. Sandman reduces power consumption by 39.38% and energy use by 33.36% while staying within 5% of SPDK's peak performance. This method was validated against Alibaba and Tencent field traces, making it suitable for cloud deployments.

2) *Shared Access and Scalability*: Device sharing remains a key challenge for kernel bypass. If each application needs exclusive NVMe queue access, multi-tenant setups become impractical. Kim et al. tackled this with BypassD [44]. It implements a split architecture where latency-critical data operations (`read()`, `write()`) run directly in user space, while metadata operations (`open()`, `append()`) are handled by the kernel. BypassD suggests IOMMU improvements to translate

virtual block addresses (VBAs) for hardware-enforced isolation between tenants, without kernel mediation on the data path.

ScalaAFA [45] addresses the scalability of user-space all-flash array engines. Based on SPDK’s lock-free approach, ScalaAFA replaces traditional locks with message-passing for permission management. It uses SSD-internal compute resources for parity computation during RAID conversion. ScalaAFA achieves $2.5\times$ the write throughput and 52.7% lower average write latency compared to the best AFA engines available.

3) *Next-Generation Bypass Platforms*: LabStor [46] introduces a modular, extensible platform for building customized I/O stacks in user space. The core abstraction is the *LabMod*, an independent code module that provides single-purpose storage functionality. These modules can be swapped, stacked, and hot-upgraded. LabStor’s modular design addresses three main issues of Linux file systems: performance degradation from workload-agnostic policies, significant software overhead from context switching and data copying, and limited flexibility due to POSIX compliance. Performance tests show the SPDK LabMod outperforming POSIX by up to 60% for 4 KB I/Os.

AUGEFS [47] develops a scalable user-space log-structured file system. It includes three main innovations: a shared and protected address space that ensures library-level file system performance with cross-process sharing; METADB, an LSM-tree-based metadata store that removes false sharing through parallel request processing; and domain-based file organization for independent space management and parallel operations. AUGIFS also introduces an asynchronous I/O stack for `fsync` that allows data block and node block I/O to overlap, which reduces synchronous write latency.

The most radical shift in conventional bypass designs is Aeolia [48]. It challenges the widely accepted idea that polling is necessary for high-performance user-space storage. Aeolia shows that most overhead previously linked to interrupts actually comes from kernel thread scheduling policies, not the interrupts themselves. By integrating user-space interrupts with MPK (hardware intra-process isolation) and `sched_ext` (eBPF-based scheduling), Aeolia creates an interrupt-based user-space storage stack that occupies a new area in the design landscape. Aeodriver outperforms `io_uring` with polling and SPDK by $8.18\times$ to $291.72\times$ when running latency-critical and compute-heavy tasks, because event-driven interrupt handling prevents CPU wastage due to polling and the issues of concurrent programming based on polling.

4) *Syscall Elimination*: Park and Volos [49] introduced userspace bypass (UB), which moves user-space instructions into the kernel to remove system call over-

head. With KPTI enabled, a single no-op system call costs 431 CPU cycles, and a `pwrite` can lower subsequent instruction IPC from 2.9 to 0.2. UB achieves 30.3–88.3% acceleration for I/O micro-benchmarks without needing any changes to application code. While it may be less effective than `io_uring` or DPDK for specific workloads, UB’s requirement for zero modifications offers a distinct advantage in the design landscape.

uFS [18] creates a user-level file system semi-microkernel based on SPDK. The uFS server carefully separates in-memory data structures and on-disk data at the inode level. This design achieves share-nothing semantics between worker threads connected to client applications through lockless ring buffers. This architecture results in a flexible, scalable, and high-performance user-space file system with POSIX-style APIs.

D. Cross-Cutting Concerns

Several themes appear across the kernel, hybrid, and bypass paths, highlighting key challenges in designing I/O stacks for ULL SSDs.

1) *The Killer Microsecond Problem*: Barroso et al. [4] identified microsecond operations as lying in a “no man’s land” between fast hardware techniques and slower software techniques. Software delays, acceptable for HDDs—such as a cumulative $75\mu\text{s}$ from RPC and TCP/IP stacks on a $2\mu\text{s}$ fabric—become the main latency issue for ULL devices.

Enberg et al. [5] quantified hardware limits: Line Fill Buffers restrict prefetches to 10 per core, and a shared on-chip interconnect queue (capacity 14) creates a bottleneck for multicore systems. Application-managed queues can get around the LFB limit but come with management costs that cap performance at 50% of the DRAM baseline.

The “I/O is faster than the CPU” position [6] argues that many OS abstractions, designed when I/O was slower than the CPU, are now outdated. It suggests a “parakernel” architecture that allocates hardware queues per process. This theoretical idea drives practical efforts across all three I/O paths.

2) *Zero-Copy and Data Movement Optimization*: Data copying between kernel and user buffers takes up 10–35% of total I/O latency with modern SSDs [50]. Kim et al. proposed a hybrid zero-copy I/O stack that checks the page cache during buffered reads: cache hits go through normally, while cache misses trigger direct transfers from the device to the user buffer, bypassing the kernel buffer completely. Delayed prefetching selectively refills the cache, minimizing conflicts with current I/O. This method cuts latency by up to 25% and boosts throughput by up to 27%.

Kim and Lee [51] further examined balancing data validity (which needs cache coherence) with zero-copy

(bypassing the cache). Their strategy delays cache filling by reading directly into user buffers on cache misses and fills the cache asynchronously when conditions allow.

3) *Latency Characterization and Hardware Acceleration*: Liu et al. [9] provided a detailed analysis of I/O latency across SATA SSDs, NVMe SSDs, Z-SSDs, and Optane devices. They found that kernel overhead rises from 8% to 36% of total latency as device speed increases (Fig. 3). Their work on mixed-workload I/O services for ULL SSDs highlighted the need for scheduling that considers workloads.

Lee et al. [3] introduced a high-performance NVMe controller with hardware acceleration, reaching 7.0 GB/s bandwidth and 1.7M IOPS, with average latencies of only 2.4 μ s (read) and 3.2 μ s (write), using 89% of PCIe bandwidth. This co-design approach shifts intensive data transfer tasks to hardware while keeping complex commands in firmware.

Since NVMe firmware can make up 86–99% of total I/O latency in PRAM and MRAM devices, hardware acceleration at the controller level offers a way to complement optimizations on the host side. FLIXR [52] shows cross-stack optimization by incorporating index data into FTL entries, removing the need for separate index I/O operations. With SSD controller processors idle for almost 70% of the time even under high I/O loads, FLIXR takes advantage of this slack to improve performance by 52.6% for TPC-C/TPC-H benchmarks with minimal changes to firmware.

4) *Virtualized and Cloud Storage*: Spool [53] addresses reliability issues in virtualized NVMe storage pools, pointing out that SPDK’s initialization process takes about 2.5 s to probe all devices and set up the Environment Abstraction Layer. During software updates, all I/O devices become unavailable, which is disruptive for cloud storage. Spool presents live-upgradable virtualized storage pools that keep I/O available during system maintenance.

Choi et al. [54] show that with NVMe SSDs delivering 5 GB/s read and 3 GB/s write speeds (which exceed 10/25GbE networks), the network has become the real bottleneck for SSD arrays, not the storage itself. Their sequential management method, which allows only one SSD to handle write requests at a time, fundamentally rethinks storage array design as individual devices are no longer the limiting factor.

V. DISCUSSION

This technical survey reveals several key themes that highlight the evolution of I/O stacks, illustrated in the optimization landscape (Fig. 7).

Convergence of kernel and bypass boundaries. The gap between “kernel I/O” and “kernel bypass” is narrowing. XRP [14] injects user-defined functions into

the kernel’s NVMe driver using eBPF. I/O Passthru [15] bypasses the block layer while staying within the kernel. Aeolia [48] implements interrupts in user space. These hybrid designs indicate that future I/O stacks will be programmable pipelines where applications choose which layers to use based on their workload needs.

Reconsidering the polling assumption. For more than a decade, polling has been the go-to method for completing ULL I/O. However, various findings challenge this idea. Aeolia [48] shows that interrupts can greatly outperform polling when running alongside compute-intensive tasks. Sandman [43] reveals that SPDK’s 100% CPU polling wastes significant energy. Harris and Altiparmak [23] uncover that traditional polling is more energy-efficient than interrupts for ULL devices, but only for dedicated cores. Calibrated interrupts [22] offer a compromise where application context informs the hardware’s completion method. Together, these results suggest that the best completion strategy depends on the workload, with no single method standing out in all cases.

Incremental versus revolutionary optimization. The ONTAP experience [40] shows that small adjustments for error-free paths can lead to over 20 $\times$ reduction in software overhead in production systems. RFUSE [16] illustrates that updating existing frameworks (like FUSE) can reach near-native file system performance. These findings argue against assuming that total redesigns are always necessary; focused improvements on specific bottlenecks can yield significant benefits with fewer adoption challenges.

Multi-tenancy challenges persist. BypassD [44] and ScalaAFA [45] improve shared access for bypass architectures, but proper multi-tenant isolation without kernel intervention remains unresolved. The IOMMU enhancements suggested by BypassD require hardware changes not yet standard in commodity processors.

Energy efficiency as a critical concern. Sandman’s 39% energy savings [43] and the polling energy analysis [23] highlight the need for I/O stack design to consider energy use alongside latency and throughput, especially for cloud services where power costs dominate.

VI. FUTURE RESEARCH DIRECTIONS

Based on gaps and trends, we propose several promising areas for future I/O stack research.

Programmable I/O stacks. The success of XRP [14] and I/O Passthru [15] suggests fully programmable I/O pipelines, allowing applications to create custom data paths from modular parts—like LabStor’s LabMod concept with eBPF safety guarantees and hardware-assisted isolation.

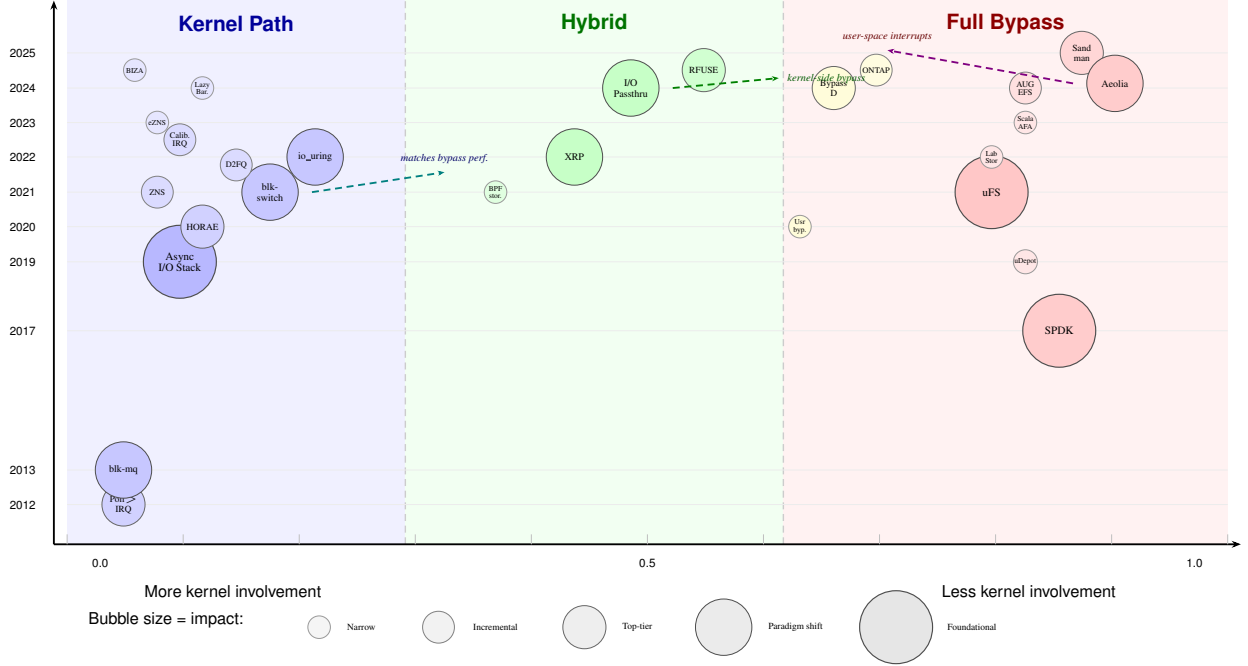


Fig. 7. Optimization landscape: I/O stack approaches positioned by kernel bypass degree (x-axis) and publication year (y-axis). Bubble size reflects impact: the three largest bubbles (SPDK, Async I/O Stack, uFS) represent foundational contributions with broad adoption. The yellow boundary zone highlights works straddling the hybrid-bypass divide. Dashed arrows indicate convergence trends.

Co-design of hardware and software for completion mechanisms. Calibrated interrupts [22] demonstrate the benefit of application-aware hardware interfaces. Future efforts should develop richer semantic interfaces that communicate workload characteristics (latency sensitivity, dependency chains, batch sizes) to device firmware and completion hardware.

Unified bypass with strong isolation. Aeolia’s [48] combination of user-space interrupts, MPK, and `sched_ext` demonstrates feasibility but requires further work on security verification, fault isolation, and integration with container/VM ecosystems.

CXL-era I/O stacks. The emergence of CXL (Compute Express Link) Type-3 memory devices will introduce byte-addressable storage with cache-coherent access, requiring fundamental rethinking of the storage stack. This represents a significant research direction beyond the scope of this review but warrants dedicated investigation.

Automated I/O stack configuration. The sensitivity of I/O performance to configuration parameters (polling mode, queue depth, scheduling policy, completion mechanism) underscores the need for automated, ML-driven stack tuning that dynamically adapts to evolving workload characteristics.

VII. CONCLUSION

This systematic review explored the evolution of I/O stack architectures to maximize the use of modern SSDs, covering 398 studies identified through PRISMA 2020-compliant methodology. Our three-path classification—Kernel, Hybrid, and Full Bypass—shows that the field is not converging toward a single architectural model but rather exploring complementary approaches, with boundaries increasingly blurred by technologies such as eBPF and NVMe passthrough.

The reviewed work indicates that software overhead in the I/O stack has become the dominant bottleneck for ULL SSDs, with the kernel storage path contributing up to 50% of end-to-end latency. Kernel path optimizations (blk-switch, D2FQ, calibrated interrupts [22]) demonstrate that carefully engineered in-kernel solutions can match or exceed bypass approaches under specific workloads. Hybrid methods (XRP [14], I/O Passthru [15], RFUSE [16]) offer compelling performance-safety trade-offs by selectively bypassing only the highest-overhead layers. Full bypass solutions continue to evolve from SPDK’s pure polling model toward energy-efficient (Sandman [43]) and interrupt-based (Aeolia [48]) designs that challenge fundamental assumptions about user-space I/O.

As storage hardware advances toward sub-microsecond latencies, the I/O stack must become more programmable, energy-aware, and adaptive to

diverse workload requirements. The convergence trends identified in this review suggest that the next generation of I/O stacks will be modular, runtime-configurable pipelines that dynamically blend kernel safety, bypass performance, and hardware assistance as required by each application context.

REFERENCES

- [1] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow *et al.*, “PRISMA 2020 explanation and elaboration: updated guidance and exemplars for reporting systematic reviews,” *BMJ*, vol. 372, p. n160, 2021.
- [2] S. Koh, C. Kim, J. Kwon, and J. Hur, “Exploring performance characteristics of ultra-low latency SSDs,” in *Proc. IEEE MAS-COTS*, 2018, pp. 35–47.
- [3] D. Lee *et al.*, “A high-performance and scalable NVMe controller featuring hardware acceleration,” *IEEE TCAD*, vol. 41, no. 5, 2022.
- [4] L. A. Barroso, M. Marty, D. Patterson, and P. Ranganathan, “Attack of the killer microseconds,” *Communications of the ACM*, vol. 60, no. 4, pp. 48–54, 2017.
- [5] P. Enberg, A. Rao, and S. Tarkoma, “Taming the killer microsecond,” *PVLDB*, vol. 11, no. 2, 2017.
- [6] —, “I/O is faster than the CPU: Let’s partition resources and eliminate (most) OS abstractions,” in *Proc. ACM HotOS*, 2019, pp. 81–87.
- [7] J. Hwang, M. Vuppapalapati, S. Peter, and A. Rachid, “Rearchitecting Linux storage stack for μ s latency and high throughput,” in *Proc. USENIX OSDI*, 2021, pp. 113–128.
- [8] J. Zhang, M. Kwon, M. Swift, and J. H. Ahn, “FlashShare: Punching through server storage stack from kernel to firmware for ultra-low latency SSDs,” in *Proc. USENIX OSDI*, 2018, pp. 477–492.
- [9] M. Liu, H. Liu, C. Ye, X. Liao, H. Jin, Y. Zhang, R. Zheng, and L. Hu, “Towards low-latency I/O services for mixed workloads using ultra-low latency SSDs,” in *Proceedings of the 36th ACM International Conference on Supercomputing*, ser. ICS ’22. ACM, 2022, pp. 1–12.
- [10] G. Lee, S. Shin, W. Song, T. J. Ham, J. W. Lee, and J. Jeong, “Asynchronous I/O stack: A low-latency kernel I/O stack for ultra-low latency SSDs,” in *Proc. USENIX ATC*, 2019, pp. 603–616.
- [11] M. Björling, J. Axboe, D. Nellans, and P. Bonnet, “Linux block I/O: introducing multi-queue SSD access on multi-core systems,” in *Proc. ACM SYSTOR*, 2013, pp. 1–10.
- [12] J. Lee, D. Kim, and S. Cho, “D2FQ: Device-direct fair queueing for NVMe SSDs,” in *Proc. USENIX FAST*, 2021, pp. 291–306.
- [13] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, “Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring,” in *Proc. ACM SYSTOR*, 2022, pp. 120–127.
- [14] Y. Zhong, H. Li, Y. J. Wu, I. Zarkadas, J. Tao, E. Mesterhazy, M. Makris, J. Yang, and A. Tai, “XRP: In-kernel storage functions with eBPF,” in *Proc. USENIX OSDI*, 2022, pp. 375–393.
- [15] K. Joshi, A. Mohan, J. Gonzalez Sondhi, A. K. Sunder, and N. Kannan, “I/O Passthru: Upstreaming a flexible and efficient I/O path in Linux,” in *Proc. USENIX FAST*, 2024.
- [16] K.-J. Cho, J. Choi, H. Kwon, and J.-S. Kim, “RFUSE: Modernizing userspace filesystem framework through scalable Kernel-Userspace communication,” in *Proceedings of the 22nd USENIX Conference on File and Storage Technologies*, ser. FAST ’24. Santa Clara, CA: USENIX Association, Feb. 2024, pp. 141–157. [Online]. Available: <https://www.usenix.org/conference/fast24/presentation/cho>
- [17] Z. Yang, J. R. Harris, B. Walker, D. Verkamp, C. Liu, C. Chang, G. Cao, J. Stern, V. Verma, and L. E. Paul, “SPDK: A development kit to build high performance storage applications,” in *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 2017, pp. 154–161.
- [18] J. Liu, A. Rebello, Y. Hao, M. Liao, A. M. Garg, S. Kannan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Scale and performance in a filesystem semi-microkernel,” in *Proc. ACM SOSP*, 2021, pp. 349–365.
- [19] H.-J. Kim, Y.-S. Lee, and J.-S. Kim, “User-level NVMe driver for low-latency flash storage,” in *Proc. IEEE NVMSA*, 2016, pp. 1–6.
- [20] B. Harris and N. Altıparmak, “Do we still need I/O schedulers for low-latency disks?” *ACM TOS*, vol. 17, no. 3, pp. 1–25, 2021.
- [21] J. Yang, D. B. Minturn, and F. Hady, “When poll is better than interrupt,” in *Proc. USENIX FAST*, 2012.
- [22] A. Yang *et al.*, “Optimizing storage performance with calibrated interrupts,” in *Proc. USENIX OSDI*, 2022.
- [23] B. Harris and N. Altıparmak, “When poll is more energy efficient than interrupt,” in *Proc. USENIX HotStorage*, 2022.
- [24] H.-J. Kim and J.-S. Kim, “A user-space storage I/O framework for NVMe SSDs in mobile smart devices,” *IEEE Transactions on Consumer Electronics*, vol. 63, no. 1, pp. 28–35, 2017.
- [25] X. Liang, L. Xia, X. Wei, H. Chen, H. Dai, S. Lu, H. Chen, and Z. Cao, “Write dependency disentanglement with HORAE,” in *Proc. USENIX OSDI*, 2020, pp. 549–565.
- [26] Y. Zhang, H. Zhang, W. Cao, X. He, D. Park, J. Choi, and S. Park, “LazyBarrier: Reconstructing android I/O stack for barrier-enabled flash storage,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, ser. ASPLOS ’24. New York, NY, USA: ACM, 2024, pp. 601–615. [Online]. Available: <https://doi.org/10.1145/3620665.3640370>
- [27] A. Papagiannis, G. Xanthakis, G. Saloustros, M. Marazakis, and A. Bilas, “Optimizing memory-mapped I/O for fast storage devices,” in *Proc. USENIX ATC*, 2020, pp. 813–827.
- [28] D. Peter, D. Bittman, H. Volos, P. Alvaro, E. Brewer, and S. Swanson, “Twizzler: A data-centric OS for non-volatile memory,” in *Proc. USENIX ATC*, 2021.
- [29] J. Son *et al.*, “Exploiting flat namespace to improve file system metadata performance on ultra-fast NVMs,” in *Proc. USENIX ATC*, 2024.
- [30] J. Park and J. Jeong, “Speculative multi-level access in LSM tree-based KV store,” *IEEE Computer Architecture Letters*, vol. 21, no. 2, pp. 145–148, 2022.
- [31] C.-F. Wu *et al.*, “UnistorFS: A union storage file system design for resource sharing between memory and storage,” *IEEE TC*, vol. 69, no. 12, pp. 1807–1820, 2020.
- [32] M. Björling, A. Aghayev, H. Holmberg, A. Ramesh, D. Le Moal, G. R. Ganger, and G. Amvrosiadis, “ZNS: Avoiding the block interface tax for flash-based SSDs,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 689–703. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/bjorling>
- [33] K. Han, H. Gwak, D. Shin, and J. Hwang, “ZNS+: Advanced zoned namespace interface for supporting in-storage zone compaction,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, Jul. 2021, pp. 147–162. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/han>
- [34] M. Björling, J. González, and P. Bonnet, “LightNVM: The Linux open-channel SSD subsystem,” in *Proc. USENIX FAST*, 2017, pp. 359–373.
- [35] J. Lee, D. Kim, K. Byun *et al.*, “eZNS: An elastic zoned namespace for commodity ZNS SSDs,” in *Proc. USENIX OSDI*, 2023, pp. 461–477.
- [36] J. Park *et al.*, “Exploring I/O management performance in ZNS with ConfZNS++,” in *Proc. USENIX FAST*, 2024.
- [37] Q. Chen *et al.*, “BIZA: Design of self-governing block-interface ZNS AFA,” in *Proc. USENIX ATC*, 2024.
- [38] J. Park *et al.*, “ZMS: Zone abstraction for mobile flash storage,” in *Proc. USENIX ATC*, 2024.
- [39] Y. Zhong, H. Li, Y. J. Wu, I. Zarkadas, J. Tao, E. Mesterhazy, M. Makris, J. Yang, and A. Tai, “BPF for storage: An exokernel-inspired approach,” in *Proc. USENIX HotOS*, 2021.

- [40] M. Curtis-Maury, R. Kesavan, B. V. R., N. Mattankot, V. Fang, Y. Trivedi, K. Mishra, and t. Q. Li, "I/O in a flash: Evolution of ONTAP to Low-Latency SSDs," in *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. Santa Clara, CA: USENIX Association, Feb. 2024, pp. 177–192. [Online]. Available: <https://www.usenix.org/conference/fast24/presentation/curtis-maury>
- [41] K. Kourtis, N. Ioannou, and I. Koltsidas, "Reaping the performance of fast NVM storage with uDepot," in *Proc. USENIX FAST*, 2019, pp. 1–15.
- [42] S. Li *et al.*, "Phoenix: Memory speed HPC I/O with NVM," in *Proc. IEEE HPCA*, 2017, pp. 121–132.
- [43] Y. Zhou, E. Xu, A. Su, J. Harris, A. Manzanares, and S. Swanson, "Sleeping with one eye open: Fast, sustainable storage with Sandman," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, ser. SOSP '25. New York, NY, USA: ACM, 2025, pp. 496–511. [Online]. Available: <https://doi.org/10.1145/3731569.3764804>
- [44] S. Yadalam, C. Alverti, V. Karakostas, J. Gandhi, and M. Swift, "BypassD: Enabling fast userspace access to shared SSDs," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '24. New York, NY, USA: ACM, 2024, pp. 35–51.
- [45] J. Lee, D. Kim *et al.*, "ScalaAFA: Constructing user-space all-flash array engine with holistic design," in *Proc. USENIX FAST*, 2023, pp. 1–17.
- [46] L. L. Lukken, S. Herbein *et al.*, "LabStor: A modular and extensible platform for developing high-performance, customized I/O stacks in userspace," in *Proc. IEEE/ACM SC*, 2022, pp. 1–14.
- [47] D. Kim, J. Lee *et al.*, "AUGEFS: A scalable userspace log-structured file system for modern SSDs," in *Proc. USENIX ATC*, 2024.
- [48] C. Li, R. Yi, Z. Zhang, J. Liu, C. Min, J. Zhang, Y. Luo, X. Wang, Z. Wang, and D. Zhou, "Aeolia: A fast and secure userspace interrupt-based storage stack," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, ser. SOSP '25. New York, NY, USA: ACM, 2025, pp. 479–495. [Online]. Available: <https://doi.org/10.1145/3731569.3764816>
- [49] Z. R. Park and H. Volos, "Userspace bypass: Accelerating syscall-intensive applications," in *Proc. USENIX OSDI*, 2020, pp. 33–49.
- [50] S.-H. Kim *et al.*, "Zero-copying I/O stack for low-latency SSDs," *IEEE TC*, vol. 71, no. 1, pp. 50–63, 2022.
- [51] S.-H. Kim, J. Lee, and J. Jeong, "Resolving the dilemma between data validity and zero-copy in the I/O stack," in *Proc. IEEE NVMSA*, 2021.
- [52] K. He *et al.*, "FLIXR: Embedding index into flash translation layer in SSDs," in *Proc. USENIX ATC*, 2022, pp. 251–266.
- [53] G. Min, S. Kim, S. Cho, and B. S. Kim, "Spool: Reliable virtualized NVMe storage pool in public cloud infrastructure," in *Proc. USENIX ATC*, 2021, pp. 97–111.
- [54] S. Kim *et al.*, "Managing array of SSDs when the storage device is no longer the performance bottleneck," in *Proc. USENIX HotStorage*, 2019.